

N044 사람이 승인하는 에이전트 만들기_해설서

Chapter01

/강의정리 챕터 01 — 사람이 승인하는 에이전트 만들기

1. 이번 강의의 핵심 주제

지금까지 만든 에이전트는

| 자료 수집 → 분류 → 조사 → 판단 → 결과물 생성

까지 자동으로 수행했습니다.

하지만 실제 업무에서는 결과를 만드는 것과 그 결과를 실제로 실행하는 것 사이에 중요한 문제가 하나 있습니다.

예를 들어 AI가 다음과 같은 결정을 내렸다고 생각해 보겠습니다.

- 고객에게 환불한다.
- 고객에게 이메일을 보낸다.
- 문서를 수정해서 저장한다.
- 콘텐츠를 게시한다.
- 고객 문의를 종결한다.
- 일정을 확정한다.

이런 작업은 한 번 실행되면 되돌리기 어렵습니다.

따라서 이번 강의의 핵심은 단순히

| “AI가 더 정확하게 판단하도록 만드는 것”

이 아니라,

| “AI가 잘못 판단했을 때 실제 실행 전에 사람이 막을 수 있도록 만드는 것”

입니다.

이것이 **Human-in-the-Loop(HITL)**, 즉 사람이 개입하는 에이전트입니다.

2. 왜 AI 정확도만 높여서는 충분하지 않을까?

가장 중요한 부분입니다.

AI의 정확도가 95%라고 가정해 보겠습니다.

하루에 200건을 처리한다면

$$200 \times 5\% = 10\text{건}$$

정도가 잘못 판단될 수 있습니다.

그렇다면 정확도를 99%까지 높이면 어떨까요?

$$200 \times 1\% = 2\text{건}$$

입니다.

99%면 상당히 정확해 보이지만, 여전히 하루 2건의 오류가 발생할 수 있습니다.

문제는 **그 2건의 성격**입니다.

예를 들어 잘못된 2건이

- 31만 원 환불
- 고객에게 잘못된 정책 안내

라면 단순히 "AI가 99% 정확하다"는 사실만으로는 문제가 해결되지 않습니다.

여기서 중요한 관점

AI 자동화에서는

| **평균적인 정확도와 개별 사고의 피해 규모는 서로 다른 문제입니다.**

AI가 99% 정확하더라도 남은 1%가 위험한 작업이라면 사람이 개입할 필요가 있습니다.

따라서 두 가지를 별도로 생각해야 합니다.

① AI의 판단 정확도를 높인다

프롬프트 개선, 모델 개선, RAG, 검증 등을 통해 오류를 줄입니다.

② 오류가 발생해도 사고로 이어지지 않게 한다

AI가 실제 실행하기 전에 사람이 확인할 수 있는 **안전장치**를 둡니다.

즉,

정확도 개선 + 실행 전 인간 승인

이라는 두 겹의 방어 구조가 필요합니다.

3. 자동화에는 세 가지 방법이 있습니다

AI에게 업무를 맡길 때 크게 세 가지 선택지가 있습니다.

방식	장점	단점
모든 건 사람이 확인	안전함	자동화 효과가 크게 줄어듦
모든 건 AI가 자동 실행	매우 빠름	사고를 막기 어려움
위험한 건만 사람 확인	속도와 안전성의 균형	위험 기준을 잘 설계해야 함

① 모든 것을 사람이 확인

200건이 들어오면 사람이 200건을 모두 확인합니다.

안전하지만 이렇게 하면

"AI가 일을 대신해 준다"

는 자동화의 장점이 상당히 사라집니다.

② 모든 것을 AI가 자동 처리

반대로 AI가 200건을 전부 판단하고 바로 실행합니다.

속도는 가장 빠릅니다.

하지만 잘못된 환불, 잘못된 이메일, 잘못된 게시물 등이 그대로 외부로 나갈 수 있습니다.

③ 위험한 것만 사람이 확인

가장 중요한 방식입니다.

예를 들어 200건 가운데

- 180건 → AI가 자동 처리
- 20건 → 사람에게 승인 요청

하는 구조입니다.

그러면 사람은 모든 일을 다시 하는 것이 아니라 **AI가 위험하다고 판단한 일부만 확인**합니다.

이것이 바로 HITL입니다.

4. HITL을 쉽게 이해하는 비유

가장 쉬운 비유는 **자동차의 자율주행**입니다.

자동차가 직선 도로를 달릴 때마다 운전자가 계속 핸들을 잡고 있다면 자율주행의 의미가 없습니다.

반대로 자동차에게 모든 상황을 100% 맡겨버리면 위험할 수 있습니다.

그래서 중요한 순간에는 사람이 개입할 수 있도록 합니다.

AI 에이전트도 비슷합니다.

일반적인 업무

| AI 판단 → 자동 실행

위험한 업무

| AI 판단 → 위험 여부 확인 → 사람 승인 → 실행

즉 사람은 AI를 대신해서 모든 일을 하는 것이 아니라,

| AI가 혼자 처리하면 위험한 순간에만 개입하는 감독자

역할을 합니다.

5. HITL의 핵심은 두 가지입니다

HITL을 구현하려면 단순히 "사람에게 확인해 주세요"라고 메시지를 보내는 것으로 끝나지 않습니다.

두 가지 기능이 필요합니다.

① 중간에 멈출 수 있어야 한다

에이전트가 작업을 하다가

| "이건 사람이 확인해야 합니다."

라고 판단하면 **실행을 잠시 중단**할 수 있어야 합니다.

② 사람의 답을 받은 뒤 다시 이어갈 수 있어야 한다

사람이

| 승인

또는

| 거절

이라고 답하면 에이전트는 처음부터 다시 시작하는 것이 아니라 **멈췄던 지점에서 이어서 작업**해야 합니다.

따라서 HITL의 기본 구조는 다음과 같습니다.

AI 작업

↓

위험도 판단

↓

일반적인 건

→ 자동 실행

위험한 건

→ 작업 중단

→ 사람에게 승인 요청

→ 승인/거절

→ 작업 재개

여기서 핵심 기술이 바로 **Pause → Human Decision → Resume** 구조입니다.

6. HITL에서 더 어려운 문제: "어떤 건을 사람에게 보낼 것인가?"

사실 HITL의 핵심은 단순히 사람에게 승인을 받는 기능이 아닙니다.

더 중요한 질문은

| "어떤 상황에서 멈춰야 하는가?"

입니다.

예를 들어 환불 업무가 있다고 하겠습니다.

모든 환불을 사람이 확인하면 자동화의 효과가 떨어집니다.

그렇다고 모든 환불을 자동으로 실행하면 위험합니다.

따라서 다음처럼 기준을 만들 수 있습니다.

예시

- 1만 원 환불 → 자동
- 5만 원 환불 → 자동
- 10만 원 이상 → 사람 승인
- 규정 해석이 애매함 → 사람 승인
- 고객 정보가 불일치 → 사람 승인
- AI 확신도가 낮음 → 사람 승인

이렇게 위험도를 기준으로 자동 처리와 사람 승인을 분리합니다.

7. 위험도는 숫자로도 설계할 수 있습니다

이번 강의에서 중요한 부분입니다.

"왠지 위험해 보이면 사람에게 보낸다"가 아니라 **객관적인 기준을 만들어야 합니다.**

예를 들어 환불 에이전트라면 다음과 같은 기준을 생각할 수 있습니다.

| 환불 금액이 일정 수준 이상인가?

| AI의 판단 확신도가 낮은가?

| 회사 규정에 예외가 있는가?

| 고객 정보가 일치하는가?

| 과거와 다른 특이한 사례인가?

이런 조건들을 조합해서 **사람 승인 기준**을 만들 수 있습니다.

즉,

AI 판단 → 위험도 평가 → 승인 필요 여부 결정

이라는 별도의 단계가 생기는 것입니다.

8. 어떤 업무에 HITL이 필요한가?

이번 강의에서 제시한 사례를 정리하면 다음과 같습니다.

업무	실제로 발생하는 위험	사람에게 보내야 하는 경우
환불·보상	돈이 나감	고액 환불, 규정 해석이 애매함
고객 답변	고객이 잘못된 내용을 읽음	확신도가 낮거나 규정 인용이 불확실함
콘텐츠 게시	잘못된 정보가 공개됨	출처가 불분명함
문서 수정	원본이 변경됨	수정 범위가 큼
문의 종결	고객 요청이 닫힘	아직 해결되지 않은 신호가 있음
일정 확정	상대방의 시간이 결정됨	일정 충돌이나 요청이 모호함

여기서 공통점을 찾아보면 매우 중요합니다.

공통점

- ① 외부에 영향을 준다.
- ② 실행 후 되돌리기 어렵다.
- ③ 모든 건의 위험도가 같지 않다.

이 세 가지 조건이 동시에 존재할 때 HITL이 특히 유용합니다.

9. AI 에이전트 설계 관점에서 무엇이 달라지는가?

기존의 자동화는 단순하게 생각하면 다음과 같습니다.

입력 → AI 처리 → 결과 → 실행

하지만 HITL을 적용하면 구조가 바뀝니다.

입력 → AI 처리 → 위험도 판단

여기서 갈라집니다.

안전한 경우

| 자동 실행 → 종료

위험한 경우

| 중단 → 사람 승인 → 실행 또는 취소 → 종료

따라서 이제 에이전트를 설계할 때부터

| "어디까지 AI가 자동으로 결정할 것인가?"

와

| "어떤 순간에는 반드시 사람에게 넘길 것인가?"

를 함께 설계해야 합니다.

이것이 중요한 이유는 HITL이 단순한 UI 기능이 아니라 **에이전트의 의사결정 구조 자체**이기 때문입니다.

10. 이번 강의에서 배우는 진짜 의미

이번 강의는 단순히 "사람이 승인 버튼을 누르는 기능"을 배우는 것이 아닙니다.

더 큰 의미는 **AI 자동화의 책임 범위를 설계하는 방법**을 배우는 것입니다.

AI를 잘 만드는 것만 생각하면

| "어떻게 하면 AI가 100% 정확하게 판단할까?"

에 집중하게 됩니다.

하지만 현실의 AI 시스템에서는 100% 정확도를 기대하기 어렵습니다.

따라서 더 현실적인 질문은

| "AI가 틀릴 수도 있다는 것을 전제로 시스템을 어떻게 설계할 것인가?"

입니다.

이 관점이 매우 중요합니다.

11. AI 개발자·AI 서비스 기획자가 알아야 하는 이유

사용자님처럼 **AI 서비스 기획과 컨설팅**을 하려는 입장에서는 특히 중요한 개념입니다.

실제 기업에서 AI를 도입할 때 고객이 원하는 것은 단순히

|"AI를 붙여주세요."

가 아닙니다.

기업은 결국 다음을 묻게 됩니다.

- AI가 잘못 판단하면 어떻게 하지?
- 누가 최종 책임을 지지?
- 돈이 나가는 업무를 AI에게 맡겨도 되나?
- 고객에게 잘못된 답변이 나가면 어떻게 하지?
- 사람이 언제 개입해야 하지?
- 모든 것을 사람이 확인해야 한다면 AI를 사용하는 의미가 있나?

이때 컨설턴트가

|"AI가 자동화합니다."

라고만 설명하는 것과

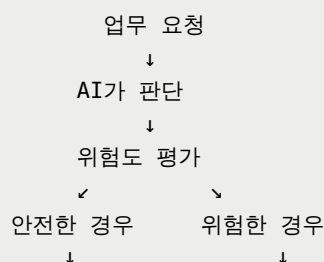
|"일반적인 건은 AI가 자동 처리하고, 위험도가 높은 건만 사람의 승인을 거친 뒤 실행하도록 설계하겠습니다."

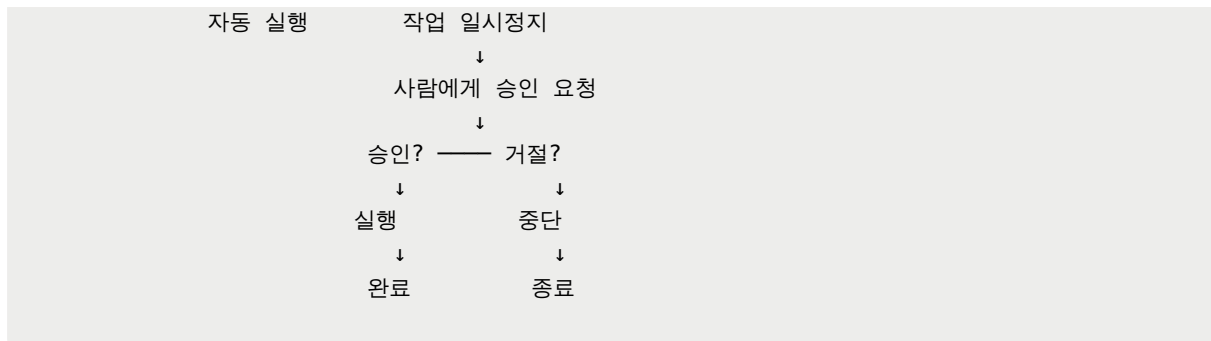
라고 설명하는 것은 서비스 설계 수준에서 큰 차이가 있습니다.

즉 HITL은 **AI 기술뿐 아니라 AI 서비스 기획·업무 프로세스 설계에서 반드시 알아야 할 개념**입니다.

12. 이번 강의의 핵심 구조

이번 챕터의 내용을 하나의 그림처럼 기억하면 쉽습니다.





핵심은 AI가 모든 것을 자동으로 실행하는 것이 아니라, 위험한 순간에 사람에게 통제권을 돌려주는 것입니다.

13. 이번 챕터에서 반드시 기억할 것

★ 1. 정확도 100%를 전제로 시스템을 만들면 안 됩니다.

AI의 정확도를 높이는 것과 AI 오류가 실제 사고로 이어지는 것을 막는 것은 서로 다른 문제입니다.

★ 2. 모든 것을 사람이 확인하는 것도 좋은 자동화가 아닙니다.

사람이 200건을 모두 확인한다면 AI가 처리해 주는 효과가 크게 줄어듭니다.

★ 3. 위험한 건만 사람에게 보내는 것이 HITL의 핵심입니다.

일반적인 건 → 자동 처리
위험한 건 → 사람 승인

★ 4. HITL은 "멈춤 + 사람 판단 + 재개" 구조입니다.

AI가 중간에 멈출 수 있어야 하고, 사람의 결정을 받은 뒤 다시 이어서 실행할 수 있어야 합니다.

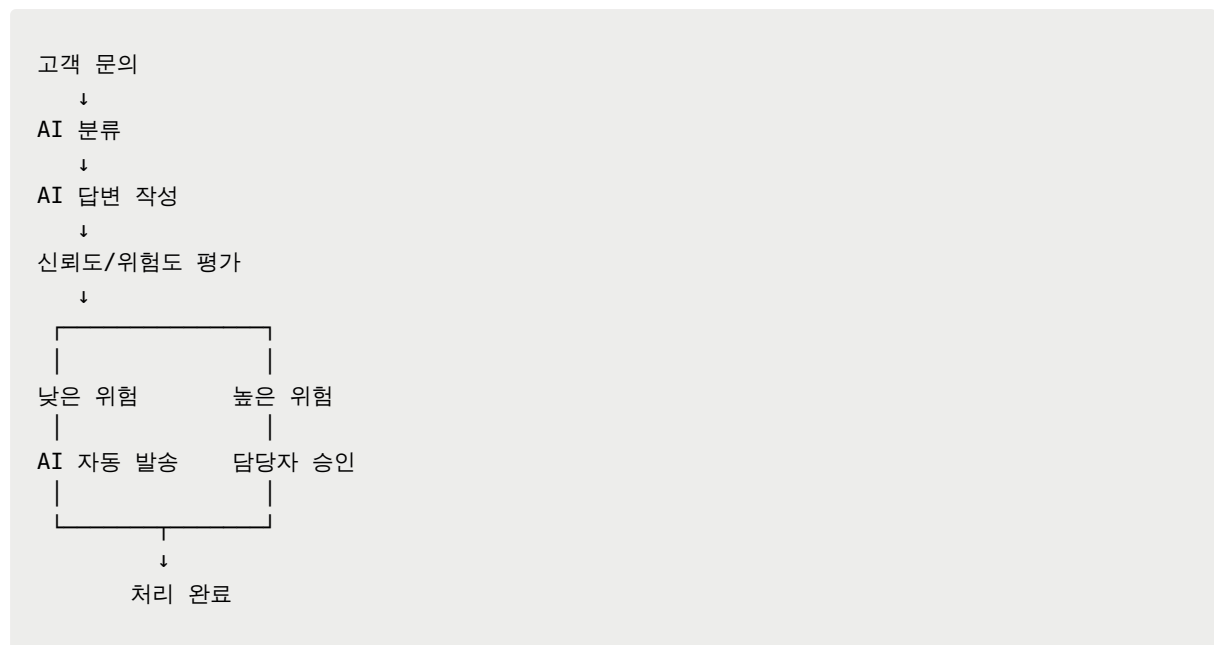
★ 5. 가장 중요한 설계는 "언제 멈출 것인가"입니다.

HITL의 품질은 승인 화면보다 사람에게 넘기는 기준에 크게 좌우됩니다.

14. 사용자님의 학습·창업 관점에서 연결하면

앞으로 사용자님이 만들게 될 AI 컨설팅이나 1인 AI 서비스에서는 HITL이 매우 중요한 설계 요소가 될 수 있습니다.

예를 들어 **기업용 AI 업무자동화 컨설팅**을 한다고 하면,



처럼 설계할 수 있습니다.

이렇게 보면 **AI 에이전트**는 단순히 "**AI가 사람의 일을 대신하는 시스템**"이 아니라, **AI와 사람이 각각 잘하는 일을 나누는 시스템**이라는 것을 알 수 있습니다.

AI는 반복적이고 빠른 판단을 담당하고, 사람은 예외·고위험·책임이 큰 판단을 담당합니다.

15. 한 문장으로 정리

HITL(Human-in-the-Loop)은 AI가 모든 일을 무조건 자동 실행하도록 하는 것이 아니라, 위험한 순간에는 작업을 멈추고 사람의 승인을 받은 뒤 다시 실행하도록 만드는 안전장치입니다.

🎯 이번 챕터의 핵심 키워드

정확도 ≠ 안전성

자동화 ≠ 무조건 무인화

위험도 기반 자동화

Pause → Human Approval → Resume

Human-in-the-Loop(HITL)

그리고 이번 실습에서는 **환불 집행 에이전트**를 사례로 이 구조를 실제로 구현하면서, **어디에서 멈출 것인지와 그 기준을 숫자로 어떻게 검증할 것인지**까지 살펴보게 됩니다.

/강의정리 챕터 02 — HITL을 두는 자리와 멈췄다가 이어가는 방법

1. 이번 강의의 핵심 주제

챕터 01에서는 왜 사람이 승인하는 AI 에이전트(HITL)가 필요한가를 배웠다면, 이번 챕터에서는 한 단계 더 들어가서

“에이전트의 어디에서 멈추고, 무엇을 저장하며, 어떻게 다시 시작하는가?”

를 배웁니다.

핵심은 다음 세 가지입니다.

- ① 실행 직전에 멈춘다 → `interrupt`
- ② 멈춘 상태를 저장한다 → 체크포인트
- ③ 사람의 답을 받아 이어간다 → `resume`

그리고 이 구조에서 매우 중요한 주의점이 하나 있습니다.

멈춘 단계는 이어서 실행되는 것이 아니라 그 단계가 처음부터 다시 실행됩니다.

이 때문에 HITL 에이전트의 설계 방식까지 달라집니다.

2. HITL은 어디에 두어야 하는가?

업무를 여러 단계로 나누어 생각해 보겠습니다.

예를 들어 환불 업무라면:

```
고객 요청 읽기
↓
규정 검색
↓
환불 가능 여부 판단
↓
환불 금액 결정
↓
사람 승인
```

↓
실제 환불 실행

여기서 앞쪽의 작업들은 잘못해도 다시 할 수 있습니다.

예를 들어 규정을 잘못 찾았다면 다시 검색하면 됩니다.

환불 여부를 잘못 판단했다면 다시 판단하면 됩니다.

하지만 **실제 돈을 보내는 순간**에는 상황이 달라집니다.

돈이 이미 나갔다면 되돌리기 어렵습니다.

따라서 HITL은 **결과가 외부로 나가기 바로 직전**에 두는 것이 핵심입니다.

가장 중요한 원칙

AI가 결과를 만드는 단계와 실제 결과를 실행하는 단계를 분리하고, 그 사이에 사람의 승인을 둡니다.

예를 들어:

[AI가 만드는 단계]

환불 판단

환불 금액 결정

↓

★ HITL ★

사람 승인

↓

[실제 실행]

환불 집행

이렇게 설계하면 AI가 잘못 판단하더라도 **실제 돈이 나가기 전에 막을 수** 있습니다.

3. 왜 "만드는 일"과 "보내는 일"을 분리해야 하는가?

이 부분은 AI 에이전트를 설계할 때 매우 중요합니다.

예를 들어 AI가 고객에게 보낼 이메일을 작성한다고 해보겠습니다.

나쁜 구조

AI가 이메일 작성

↓

AI가 잘못 작성하면 그대로 고객에게 전달됩니다.

좋은 구조

AI가 이메일 작성
↓
사람 승인
↓
이메일 발송

즉,

■ **생성(Prepare)** 과 **실행(Execute)** 을 분리합니다.

이렇게 하면 그 사이에 HITL을 넣을 수 있습니다.

이 원리는 환불뿐 아니라

- 이메일 발송
- 콘텐츠 게시
- 외부 시스템 등록
- 문서 수정
- 일정 확정
- 고객 문의 종결

등에도 똑같이 적용할 수 있습니다.

4. HITL의 실제 동작 구조

HITL을 쉽게 표현하면 다음과 같습니다.

AI 작업
↓
검토 단계
↓
interrupt
↓
———— 멈춤 ————
↓
사람이 승인/반려

↓
resume
↓
검토 단계부터 다시 시작
↓
승인 → 실행
반려 → 종료/반려처리

여기서 중요한 것은 **멈췄다가 나중에 이어간다는 것**입니다.

AI 프로그램이 처음부터 다시 모든 일을 하는 것이 아닙니다.

멈췄던 요청의 상태를 저장해 두었다가 **그 요청의 다음 작업을 이어가는 것**입니다.

5. **interrupt** — 에이전트를 멈추는 기능

interrupt 는 말 그대로 "**잠깐 멈춰!**"라는 의미입니다.

예를 들어 환불 에이전트가 다음과 같은 정보를 가지고 있다고 생각해 보겠습니다.

요청:
코트를 한 달 입었는데 어깨 솔기가 터졌습니다.

금액:
310,000원

승인하면:
310,000원이 계좌로 나갑니다.

AI가 여기서 **interrupt** 를 만나면 작업을 멈춥니다.

그리고 이 정보가 사람에게 전달됩니다.

즉 **interrupt** 안에 무엇을 넣느냐가 사실상 **사람이 보는 승인 화면의 내용**이 됩니다.

따라서 좋은 승인 정보에는

- 어떤 요청인가?
- 얼마인가?
- 왜 처리하려는가?
- 승인하면 어떤 일이 발생하는가?

등이 명확하게 들어 있어야 합니다.

6. **resume** — 사람의 답을 받아 다시 시작하기

멈춘 뒤 사람이 판단합니다.

예를 들어

```
approve
```

또는

```
reject
```

라고 답할 수 있습니다.

이 답을 **resume** 으로 에이전트에게 전달합니다.

그러면 에이전트는 그 답을 보고 다음 행동을 결정합니다.

```
approve
```

↓

```
환불 집행
```

```
reject
```

↓

```
반려 처리
```

즉,

| **interrupt** = 멈추기

| **resume** = 사람의 답으로 다시 시작하기

라고 기억하면 됩니다.

7. **thread_id** — 여러 요청을 섞이지 않게 하는 번호

실제 서비스에서는 환불 요청이 하나만 들어오지 않습니다.

동시에 여러 요청이 들어올 수 있습니다.

예를 들어

```
R1 → 승인 대기
R2 → 승인 대기
R3 → 승인 대기
R4 → 승인 대기
```

이런 상황에서 R3에 대한 승인 답변이 R4에 적용되면 큰 문제가 됩니다.

그래서 각각의 요청에 ****고유한 thread_id****를 붙입니다.

```
R3 → thread_id = R3
R4 → thread_id = R4
```

그러면 에이전트는

| "이 답변은 R3 요청에 대한 것이구나."

라고 구분할 수 있습니다.

쉽게 비유하면

은행 창구에서 고객마다 **대기번호표**를 받는 것과 같습니다.

사람이 여러 명 기다리고 있어도

| "R3 고객의 업무를 계속하세요."

라고 정확하게 지정할 수 있는 것입니다.

8. 체크포인트란 무엇인가?

이번 챕터에서 가장 중요한 개념 중 하나입니다.

에이전트가 멈췄다면 나중에 다시 이어서 실행해야 합니다.

그런데 프로그램 입장에서는 이미 실행이 중단됐습니다.

그러면 이런 질문이 생깁니다.

| "지금까지 무엇을 했지?"

| "어떤 값을 가지고 있었지?"

| "다음에 어디서 시작해야 하지?"

이 정보를 저장해 두는 것이 **체크포인트(Checkpoint)**입니다.

9. 체크포인트를 쉽게 이해하는 비유

책을 읽다가 잠시 중단한다고 생각해 보겠습니다.

책갈피가 없다면 다시 돌아왔을 때

|"내가 몇 페이지까지 읽었지?"

라고 찾아야 합니다.

하지만 책갈피가 있다면

|"127페이지까지 읽었고, 다음에는 128페이지부터 읽으면 된다."

라고 바로 이어갈 수 있습니다.

에이전트의 체크포인트도 똑같습니다.

체크포인트에는 크게 두 가지가 저장됩니다.

① 지금까지 계산된 상태

예:

- 고객 요청
- 환불 여부
- 환불 금액
- 사람의 결정

② 다음에 실행할 단계

예:

다음 단계 = review

따라서 **resume** 이 들어오면 저장된 상태를 불러오고 다음 실행 위치를 찾아서 계속 진행할 수 있습니다.

10. 체크포인트가 없으면 왜 안 되는가?

HITL의 핵심 흐름은

| 멈춤 → 기다림 → 재개

입니다.

그런데 멈춘 뒤 아무것도 저장하지 않는다면 재개할 방법이 없습니다.

따라서

| **interrupt** + **체크포인트** + **resume**

은 서로 연결되어 있습니다.

```
interrupt
↓
체크포인트 저장
↓
사람의 응답 대기
↓
resume
↓
체크포인트 복원
↓
계속 실행
```

이 구조를 하나의 세트로 기억하는 것이 좋습니다.

11. 체크포인트의 종류

저장 장소에 따라 여러 종류가 있습니다.

체크포인트	저장 장소	프로그램 종료 후
InMemorySaver	메모리	사라짐
SqliteSaver	파일	유지됨
PostgresSaver	데이터베이스	유지됨

InMemorySaver

오늘 실습처럼 간단하게 구조를 테스트할 때 사용합니다.

하지만 프로그램을 켜다 켜면 기록이 사라집니다.

SqliteSaver

파일에 저장하기 때문에 프로그램을 다시 실행해도 기록을 유지할 수 있습니다.

PostgresSaver

실제 서비스처럼 여러 서버가 사용하거나 장기간 상태를 관리해야 할 때 적합합니다.

따라서 실습에서는

```
app = g.compile(checkpointer=InMemorySaver())
```

처럼 간단하게 시작합니다.

하지만 실제 서비스에서는 "사람이 몇 시간 뒤에 승인할 수도 있다"는 상황을 고려해 지속적인 저장소를 사용해야 합니다.

12. 체크포인트에서 특히 중요한 것

체크포인트를 단순히

| "중간 데이터를 저장하는 곳"

이라고만 이해하면 부족합니다.

더 정확하게는

| "에이전트가 어디까지 진행되었고, 어떤 상태이며, 어디서 다시 시작해야 하는지를 기억하는 기록"

입니다.

그래서 `thread_id` 와 함께 생각해야 합니다.

```
R3
└─ R3의 체크포인트

R4
└─ R4의 체크포인트
```

서로 섞이지 않습니다.

13. `get_state` 와 `get_state_history`

체크포인트에 무엇이 들어 있는지도 확인할 수 있습니다.

get_state

특정 `thread_id`의 가장 최근 상태를 확인합니다.

쉽게 말하면

| "R3의 현재 상태가 뭐야?"

입니다.

get_state_history

지금까지 저장된 **전체 상태의 흐름**을 확인합니다.

즉,

| "R3이 처음부터 지금까지 어떻게 진행되어 왔어?"

를 볼 수 있습니다.

따라서 디버깅이나 에이전트 동작 분석에도 매우 유용합니다.

14. 중요한 주의점 — 멈춘 단계는 처음부터 다시 실행된다

이번 챕터에서 **실무적으로 가장 중요한 내용**입니다.

많은 분들이 HITL을 이렇게 생각하기 쉽습니다.

| "중간에 멈췄으니까 그 줄 다음부터 실행되겠지."

하지만 그렇지 않습니다.

`interrupt`가 포함된 단계 자체가 다시 실행됩니다.

예를 들어:

review 단계

- ① 고객 정보 확인
- ② 고객에게 알림 보내기
- ③ `interrupt()`

여기서 `interrupt()`에서 멈췄다고 해보겠습니다.

다시 `resume`하면

review 단계

↓

- ① 고객 정보 확인
- ② 고객에게 알림 보내기 ← 다시 실행
- ③ interrupt 이후 진행

처럼 될 수 있습니다.

15. 그래서 어떤 사고가 발생할 수 있는가?

예를 들어 `interrupt` 전에

| "고객님의 요청을 검토하고 있습니다."

라는 문자를 보내도록 만들었다고 하겠습니다.

처음 실행:

문자 발송

↓

`interrupt`

↓

멈춤

사람이 승인:

resume

↓

review 단계 재실행

↓

문자 발송

결과적으로 고객은 같은 문자를 두 번 받을 수 있습니다.

이것은 단순한 코딩 문제가 아니라 에이전트 구조 설계 문제입니다.

16. 그래서 지켜야 할 설계 원칙

가장 중요한 원칙은 이것입니다.

| 사람에게 묻는 단계에서, `interrupt` 전에 외부로 나가는 작업을 하지 않는다.

즉 승인 단계에서는

- 외부 이메일 발송 ❌
- 문자 발송 ❌
- 결제 실행 ❌
- 외부 시스템 변경 ❌

같은 작업을 하지 않는 것이 좋습니다.

대신 다음처럼 분리합니다.

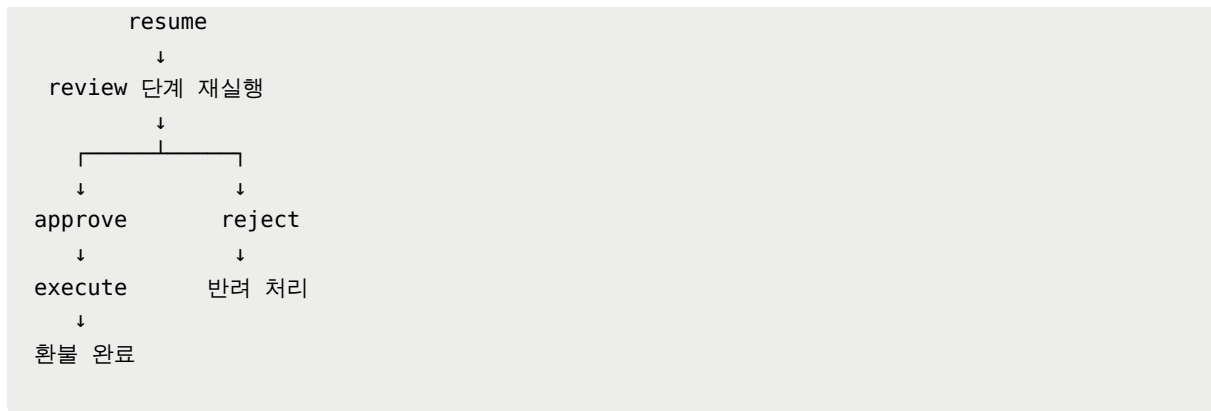
```
[검토 단계]
환불 정보 준비
↓
interrupt
↓
사람 승인
↓
[실행 단계]
환불 집행
```

이렇게 하면 **review**가 다시 실행되더라도 외부에 부작용을 일으킬 가능성을 줄일 수 있습니다.

17. 이번 챕터의 전체 구조

이번 내용을 하나의 흐름으로 정리하면 다음과 같습니다.

```
고객 요청
↓
AI 판단
↓
환불 정보 준비
↓
[review]
[interrupt]
↓
체크포인트 저장
↓
사람 대기
↓
approve / reject
↓
```



그리고 모든 요청은

| `thread_id`

로 구분합니다.

18. 이번 챕터가 중요한 이유

이번 챕터를 통해 AI 에이전트에 대한 관점이 한 단계 바뀝니다.

처음에는 에이전트를

| "AI가 알아서 일을 처리하는 프로그램"

이라고 생각하기 쉽습니다.

하지만 실제 업무용 에이전트는 더 정확하게

| "AI가 업무를 진행하다가 필요한 순간 사람에게 통제권을 넘기고, 사람의 판단을 받은 뒤 다시 작업을 이어가는 시스템"

이라고 이해해야 합니다.

특히 기업용 AI에서는 자동화율만 높이는 것보다

| 어디까지 자동화하고 어디에서 인간의 통제를 남겨둘 것인가

가 매우 중요한 설계 문제가 됩니다.

19. 사용자님의 AI 컨설팅 관점에서 기억할 것

앞으로 기업의 업무자동화를 설계할 때는 단순히

| "이 업무를 AI로 자동화할 수 있습니다."

라고 생각하면 안 됩니다.

다음 질문까지 해야 합니다.

- ① 어디까지 AI가 자동으로 처리할 것인가?
- ② 어디에서 외부 영향이 발생하는가?
- ③ 외부 영향 직전에 사람의 승인을 둘 필요가 있는가?
- ④ 승인 대기 중인 상태를 어디에 저장할 것인가?
- ⑤ 여러 요청을 어떻게 구분할 것인가?
- ⑥ 재개할 때 중복 실행되는 작업은 없는가?

이 질문까지 할 수 있다면 단순한 AI 도구 활용을 넘어 업무용 AI 에이전트 설계를 할 수 있게 됩니다.

20. 이번 챕터의 핵심 정리

★ 반드시 기억할 7가지

- 1. HITL은 결과가 외부로 나가기 직전에 듭니다.
- 2. 생성 단계와 실행 단계를 분리해야 합니다.
- 3. `interrupt` 는 에이전트를 멈추는 기능입니다.
- 4. `resume` 은 사람의 답을 받아 작업을 다시 이어가는 기능입니다.
- 5. 체크포인트는 멈춘 에이전트의 상태와 재개 위치를 저장합니다.
- 6. `thread_id` 는 여러 요청의 상태가 섞이지 않게 구분합니다.
- 7. `interrupt` 가 있는 단계는 재개할 때 처음부터 다시 실행되므로 외부 부작용을 넣으면 안 됩니다.

🎯 가장 중요한 한 문장

HITL 에이전트는 `interrupt` 로 위험한 실행 직전에 멈추고, 체크포인트에 상태를 저장한 뒤, 사람의 `resume` 응답을 받아 해당 단계부터 다시 실행하여 안전하게 업무를 이어가는 구조입니다.

/강의정리 챕터 03 — 멈춤 기준 정하기

1. 이번 강의의 핵심 주제

챕터 02에서는 **어디에서 멈추고(`interrupt`), 어떻게 다시 시작하는지(`resume`)**를 배웠습니다.

이번에는 더 중요한 질문으로 넘어갑니다.

“모든 요청을 멈출 것인가? 아니면 어떤 요청만 멈출 것인가?”

HITL은 사람에게 승인을 받는 기능만 만든다고 끝나는 것이 아닙니다.

언제 사람에게 넘길지를 결정하는 기준이 제대로 설계되어야 실제 자동화가 가능합니다.

핵심은 다음입니다.

멈춤 기준 → 개입률 → 놓침 → 헛멈춤 → 기준 비교·검증

2. 모든 건을 멈추면 자동화가 아니다

환불 요청이 5건 들어왔다고 생각해 보겠습니다.

건	금액	요청 내용	사람 확인
R1	12,000원	택배 박스가 찢어져 도착	✗
R2	28,000원	색상이 상세페이지와 다름	✗
R3	310,000원	한 달 사용 후 코트 어깨 솔기 파손	✓
R4	45,000원	사이즈 교환 대신 환불	✓
R5	9,000원	단순히 마음에 들지 않음	✓

여기서 중요한 것은 사람이 봐야 하는 이유가 모두 같지 않다는 것입니다.

R3

금액이 크기 때문에 사람이 확인합니다.

R5

금액은 9,000원으로 작지만 단순 변심이라는 이유 때문에 확인해야 합니다.

즉,

| 위험은 하나의 기준으로만 결정되지 않습니다.

3. 금액 기준 하나만 사용하면 생기는 문제

예를 들어 다음과 같은 기준을 만들었다고 해보겠습니다.

| "30,000원보다 큰 환불은 사람이 확인한다."

그러면 R3은 잡힙니다.

R3 = 310,000원
→ 사람 확인

하지만 R5는 잡히지 않습니다.

R5 = 9,000원
→ 자동 처리

그런데 R5 역시 사람이 확인해야 하는 건이었습니다.

즉,

| 금액 기준만으로는 모든 위험을 잡을 수 없습니다.

여기서 중요한 AI 에이전트 설계 원칙을 배울 수 있습니다.

위험의 종류가 다르면 기준도 여러 종류가 필요합니다.

예를 들어

금액이 큰가?
+
요청 내용이 위험한가?
+
AI의 판단 확신도가 낮은가?
+
규정 해석이 애매한가?

처럼 여러 조건을 사용할 수 있습니다.

4. "위험하면 멈춘다"를 컴퓨터가 이해할 수 있게 만들어야 한다

사람끼리는

| "금액이 크면 멈춰."

라고 말해도 대충 이해할 수 있습니다.

하지만 에이전트에게는 이런 표현이 너무 모호합니다.

예를 들어

| "금액이 크면"

이라는 조건에서 **얼마가 큰 금액인지** 알 수 없습니다.

대신 이렇게 바뀌야 합니다.

| "환불 금액이 30,000원을 초과하면 멈춘다."

마찬가지로

| "이상한 요청이면 멈춘다."

가 아니라

| "요청 내용에 '단순 번심' 또는 '마음에 안'이라는 표현이 포함되면 멈춘다."

처럼 만들어야 합니다.

즉 사람이 사용하는 자연어 기준을 **계산 가능한 규칙**으로 바꾸는 것입니다.

5. 왜 계산 가능한 기준이 중요한가?

이렇게 만들어야 기준을 바꿔가며 실험할 수 있습니다.

예를 들어:

기준 A

환불 금액 > 30,000원

기준 B

환불 금액 > 50,000원

기준 C

환불 금액 > 30,000원
또는 단순 변심 표현 포함

이렇게 기준을 변경하면서 같은 데이터를 통과시켜 볼 수 있습니다.

그러면

|"어떤 기준이 사람의 일을 가장 적게 만들면서 위험한 건을 잘 잡는가?"

를 비교할 수 있습니다.

이것이 이번 실습의 핵심입니다.

6. 멈춤 기준을 평가하는 세 가지 숫자

기준을 만들었다고 바로 적용해서는 안 됩니다.

반드시 실제 데이터에 적용해서 결과를 확인해야 합니다.

이번 강의에서는 세 가지를 봅니다.

① 개입률

전체 요청 가운데 **사람에게 넘어온 비율**입니다.

예를 들어 100건 중 20건이 사람에게 넘어왔다면

|" **개입률 = 20%**

입니다.

개입률이 낮을수록 일반적으로 사람이 처리해야 하는 일이 줄어듭니다.

하지만 무조건 낮은 것이 좋은 것은 아닙니다.

개입률을 지나치게 낮추면 위험한 요청까지 자동으로 통과할 수 있기 때문입니다.

7. ② 놓침 — 가장 중요한 위험

놓침(Miss)은

|" **사람이 확인했어야 하는데 자동으로 처리되어 버린 건**

입니다.

이번 예에서는 R5가 대표적인 사례입니다.

금액 기준만 적용하면

R5 = 9,000원
→ 자동 처리

하지만 실제로는 사람이 봐야 했습니다.

이것이 바로 **놓침**입니다.

놓침이 위험한 이유는 사람이 승인 목록에서 발견할 수 없다는 것입니다.

사람에게 올라오지 않았기 때문에

|"우리가 놓쳤다."

는 사실조차 모를 수 있습니다.

따라서 HITL에서 놓침은 **가장 중요한 오류 중 하나**입니다.

8. ③ 헛멈춤 — 불필요한 사람 개입

반대로 **헛멈춤(False Positive)**도 있습니다.

원래 AI가 자동으로 처리해도 되는 건인데 사람에게 넘어오는 경우입니다.

예를 들어

12,000원
단순한 배송 포장 손상

처럼 위험도가 낮은 요청까지 전부 사람에게 보내면 담당자가 확인해야 합니다.

이런 건이 계속 쌓이면 담당자는

|"또 승인해 주세요."

라는 요청을 계속 처리하게 됩니다.

그러면 사람이 내용을 깊게 검토하지 않고

| **그냥 승인 버튼만 누르는 습관**

이 생길 수 있습니다.

따라서 헛멈춤이 너무 많아도 좋은 HITL이 아닙니다.

9. 개입률·농침·헛멈춤의 관계

세 가지를 같이 생각해야 합니다.

개입률이 너무 높으면

사람의 일이 많아집니다.

개입률이 너무 낮으면

농침이 늘어날 가능성이 커집니다.

헛멈춤이 너무 많으면

사람이 불필요한 요청을 계속 검토하게 됩니다.

따라서 좋은 기준은 단순히

|"사람에게 적게 보내는 기준"

이 아닙니다.

목표는

|"위험한 건은 최대한 잡으면서, 사람에게 불필요한 일을 최소화하는 기준"

입니다.

10. 이것을 쉽게 비유하면 공항 보안검색과 비슷합니다

모든 승객을 아주 강하게 검사하면 안전성은 높일 수 있지만 시간이 오래 걸립니다.

반대로 거의 검사하지 않으면 빠르지만 위험을 놓칠 수 있습니다.

그래서 현실에서는 위험도가 높은 사람이나 상황을 더 집중적으로 확인하는 방식이 필요합니다.

HITL도 비슷합니다.

모든 요청



위험도 판단



낮은 위험 → 자동 처리
높은 위험 → 사람 확인

즉 HITL은 사람의 시간을 가장 중요한 곳에 사용하는 구조라고 볼 수 있습니다.

11. 기준이 너무 느슨하면

예를 들어 아예 멈춤 기준을 적용하지 않는다면

5건 모두 자동 처리

됩니다.

개입률은 0%입니다.

사람이 할 일은 없습니다.

매우 효율적으로 보이지만, 사람이 봐야 할 3건까지 자동으로 빠져나갑니다.

즉,

| 자동화율은 높지만 안전성이 떨어집니다.

12. 반대로 기준이 너무 엄격하면

이번에는 모든 요청을 사람에게 보내겠습니다.

5건 → 5건 모두 사람 확인

놓침은 0이 될 수 있습니다.

하지만 AI가 처리할 수 있는 일까지 사람이 모두 확인합니다.

결국

| AI 자동화를 도입했지만 사람이 다시 모든 일을 확인하는 구조

가 됩니다.

따라서 이것 역시 좋은 설계라고 보기 어렵습니다.

13. 중요한 것은 "균형점"입니다

HITL의 핵심은 결국 자동화와 안전성 사이의 균형입니다.

이를 간단하게 표현하면:

기준을 느슨하게

↓

개입률 ↓

놓침 ↑

↓

사람의 일 ↓

하지만 위험 ↑

기준을 엄격하게

↓

개입률 ↑

놓침 ↓

↓

안전 ↑

하지만 사람의 일 ↑

따라서 기준을 여러 가지로 바꾸어 보면서 적절한 지점을 찾아야 합니다.

14. 이번 실습에서 왜 여러 기준을 비교하는가?

강의에서는 동일한 5건을 가지고 여섯 가지 멈춤 기준을 비교합니다.

핵심은 기준 자체를 외우는 것이 아닙니다.

중요한 것은 실험하는 사고방식입니다.

예를 들어:

기준 A → 개입률 / 놓침 / 헛멈춤

기준 B → 개입률 / 놓침 / 헛멈춤

기준 C → 개입률 / 놓침 / 헛멈춤

...

이렇게 비교합니다.

그리고

|"어떤 기준이 가장 좋은가?"

를 데이터로 판단합니다.

15. "금액 3만 원 초과 + 표현"의 의미

특히 강의에서 중요한 사례가 이것입니다.

단순히

| 금액 > 30,000원

만 사용하면 R5를 놓칩니다.

하지만

| 금액 > 30,000원 OR 특정 위험 표현 포함

처럼 조건을 추가하면 R3과 R5를 함께 잡을 수 있습니다.

즉 한 가지 위험 요소만 보는 것이 아니라 서로 다른 위험 신호를 조합하는 것입니다.

이것은 실제 AI 에이전트에서도 매우 중요합니다.

예를 들어 고객 상담 에이전트라면

금액이 큰가?
OR
법적 표현이 있는가?
OR
고객이 강한 불만을 표시했는가?
OR
AI 확신도가 낮은가?

등을 조합할 수 있습니다.

16. 다만 작은 테스트 데이터만 보고 "완벽하다"고 생각하면 안 됩니다

5건을 대상으로 테스트했는데

| 놓침 = 0

| 헛멈춤 = 0

이 나왔다고 해서 실제 업무에서도 완벽하다고 할 수는 없습니다.

왜냐하면 5건밖에 테스트하지 않았기 때문입니다.

실제 업무에서는 훨씬 다양한 상황이 존재합니다.

따라서 이번 실습의 0이라는 결과는

|"이 5건에서는 0이었다."

라는 의미이지

|"실제 업무에서도 항상 0이다."

라는 의미가 아닙니다.

이 차이를 이해하는 것이 매우 중요합니다.

17. 이번 강의의 진짜 핵심 — 기준은 "정하는 것"보다 "검증하는 것"

HITL 기준을 만들 때 흔히 하는 실수는

|"금액이 3만 원 이상이면 사람이 확인하도록 하겠습니다."

라고 정한 뒤 바로 개발하는 것입니다.

하지만 실제로는 다음 순서가 더 좋습니다.

- ① 위험 요소 찾기
- ↓
- ② 멈춤 기준 만들기
- ↓
- ③ 과거 데이터에 적용
- ↓
- ④ 개입률 계산
- ↓
- ⑤ 놓침 계산
- ↓
- ⑥ 헛멈춤 계산
- ↓
- ⑦ 여러 기준 비교
- ↓
- ⑧ 기준 선택
- ↓
- ⑨ 실제 운영 후 다시 검증

즉 HITL 기준도 하나의 실험 대상입니다.

18. AI 에이전트 개발에서 중요한 이유

이 부분은 사용자님처럼 **AI 서비스 기획과 컨설팅**을 하려는 경우 특히 중요합니다.

기업에서

|"환불 업무를 AI로 자동화해주세요."

라고 요청했다고 생각해 보겠습니다.

단순한 개발자는

|"환불 여부를 판단하는 AI를 만들겠습니다."

라고 접근할 수 있습니다.

하지만 업무 프로세스 관점에서는 더 많은 질문이 필요합니다.

| 어떤 환불은 자동 처리할 것인가?

| 어떤 환불은 사람이 확인할 것인가?

| 사람에게 넘어오는 비율은 몇 %인가?

| 실제로 사람이 봐야 할 건을 놓치고 있지는 않은가?

| 불필요한 승인 요청은 얼마나 발생하는가?

이 질문을 통해 **AI 모델의 성능뿐 아니라 업무 프로세스 전체의 성능**을 측정할 수 있습니다.

19. 사용자님의 AI 컨설팅과 연결하면

앞으로 기업의 AI 자동화 컨설팅을 한다면 다음과 같은 프레임으로 업무를 분석할 수 있습니다.

Step 1. 업무를 나눈다

접수 → 분석 → 판단 → 승인 → 실행

Step 2. 위험한 지점을 찾는다

돈이 나가는가?
고객에게 전달되는가?

외부 시스템이 변경되는가?
되돌릴 수 없는가?

Step 3. 사람의 개입 기준을 만든다

금액
+ 고객 유형
+ 규정 예외
+ AI 확신도
+ 특정 키워드

Step 4. 실제 데이터로 검증한다

개입률
놓침
헛멈춤

이렇게 접근하면 단순히 "AI를 도입하는 컨설팅"이 아니라

| "AI + 업무 프로세스 + 인간 통제 구조를 설계하는 컨설팅"

으로 발전할 수 있습니다.

20. 이번 챕터에서 반드시 기억할 것

★ 1. 모든 건을 멈추는 것이 HITL의 목적이 아닙니다.

그렇게 하면 사람이 AI를 대신하게 됩니다.

★ 2. 하나도 멈추지 않는 것도 HITL이 아닙니다.

위험한 요청까지 자동으로 외부로 나갈 수 있습니다.

★ 3. 위험은 여러 종류입니다.

금액이 큰 위험과 금액은 작지만 내용이 위험한 경우를 모두 고려해야 합니다.

★ 4. 기준은 계산 가능하게 만들어야 합니다.

| "금액이 크면" ✖

| "30,000원 초과이면" ○

"이상한 요청이면" ✖

"'단순 변심'이라는 표현이 포함되면" ○

★ 5. 세 가지 지표를 봅니다.

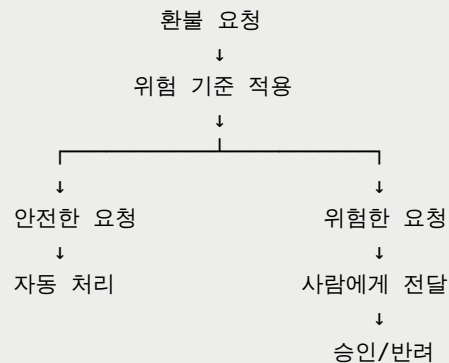
개입률 / 놓침 / 헛멈춤

특히 놓침은 가장 위험한 실수가 될 수 있습니다.

★ 6. 기준을 정한 뒤 반드시 실제 데이터로 검증합니다.

좋아 보이는 기준을 만드는 것보다 실제로 어떤 결과를 내는지 확인하는 것이 중요합니다.

21. 한눈에 보는 전체 구조



그리고 이 기준을 평가합니다.



결국 이번 챕터의 핵심은 이것입니다.

HITL의 진짜 실력은 "사람에게 승인받는 기능"이 아니라, "어떤 일을 사람에게 넘길 것인지 정확하게 결정하는 기준"에서 나옵니다.

🎯 한 문장 요약

좋은 HITL은 모든 일을 멈추는 것이 아니라, 위험한 건은 놓치지 않으면서 사람에게 불필요한 검토를 최소화하도록 멈춤 기준을 데이터로 검증하는 자동화 구조입니다.

Chapter04

/강의정리 챕터 04 — 사람이 빠르게 판단할 수 있는 승인 화면 만들기

1. 이번 강의의 핵심 주제

앞의 챕터에서 언제 멈출지를 정했다면, 이번에는 멈춘 다음 사람이 어떻게 승인할 것인지를 설계합니다.

HITL은 단순히

“AI가 멈추고 사람에게 승인 버튼을 보여준다.”

로 끝나지 않습니다.

사람이 승인하기 위해 다시 여러 시스템을 찾아봐야 한다면 자동화의 효과가 크게 떨어집니다.

따라서 이번 챕터의 핵심 목표는 아주 명확합니다.

담당자가 승인 화면 하나만 보고 10초 안에 판단할 수 있도록 만든다.

즉 HITL에서 사람은 AI의 일을 다시 하는 것이 아니라 AI가 정리해 놓은 정보를 바탕으로 최종 판단만 하는 구조를 만들어야 합니다.

2. 나쁜 승인 화면과 좋은 승인 화면

✗ 나쁜 화면

요청 ID: R3

판정: 환불 승인

승인하시겠습니까?

[승인] [반려]

겉으로는 아주 간단합니다.

하지만 담당자는 아무것도 판단할 수 없습니다.

담당자는 다시

1. 주문 시스템을 열고
2. 구매 금액을 확인하고
3. 구매 날짜를 확인하고
4. 고객 요청 원문을 찾아보고
5. 환불 규정을 검색하고
6. AI가 왜 환불이라고 판단했는지 확인해야 합니다.

결국

AI가 앞에서 정보를 정리해 놓았는데 사람은 다시 처음부터 조사해야 하는 상황

이 됩니다.

이렇게 되면 자동화로 절약한 시간이 사라집니다.

3. 좋은 승인 화면은 무엇이 다른가?

좋은 화면은 담당자가 판단에 필요한 핵심 정보를 한곳에 모아줍니다.

예를 들어:

고객 요청:
"코트를 한 달 입었는데 어깨 솔기가 터졌습니다."

금액:
310,000원

AI 판정:
환불 승인

근거 규정:
"제조 불량은 구매 후 1년 이내 전액 환불"
(환불규정 3조 2항)

멈춘 이유:
30,000원 초과

통과시키면:
310,000원이 고객 계좌로 환불됩니다.

[승인]
[금액 수정 후 승인]
[반려]
[다시 판정]

이 화면에는 중요한 정보가 모두 들어 있습니다.

4. 승인 화면에 들어가야 할 5가지

이번 강의에서 가장 중요한 부분입니다.

좋은 HITL 화면에는 크게 **5가지 정보**가 필요합니다.

① 요청 원문

고객이 실제로 무엇을 요청했는지 보여줍니다.

AI가 요약한 내용만 보여주는 것보다 원문을 확인할 수 있어야 합니다.

② 핵심 정보

예:

- 환불 금액
- 주문일
- 상품
- 고객 정보
- 관련 날짜

처럼 판단에 필요한 핵심 데이터를 보여줍니다.

③ AI의 판정과 근거

단순히

|"AI 판정: 환불 승인"

만 보여주면 부족합니다.

왜 그렇게 판단했는지도 보여줘야 합니다.

예:

| 제조 불량은 구매 후 1년 이내 전액 환불

그리고 가능하면 어떤 규정의 어떤 조항인지까지 보여줍니다.

④ 왜 사람에게 넘어왔는가

예:

| **멈춘 이유: 환불 금액 30,000원 초과**

이 정보가 있어야 담당자가

| "아, AI가 이 조건 때문에 나에게 넘겼구나."

라고 바로 이해할 수 있습니다.

⑤ 승인하면 어떤 일이 일어나는가

이 부분이 매우 중요합니다.

단순히

| [승인]

이라고 쓰는 것보다

| **"승인하면 310,000원이 고객 계좌로 환불됩니다."**

라고 알려주는 것이 훨씬 안전합니다.

사람이 버튼을 눌렀을 때 **실제로 어떤 일이 발생하는지** 명확하게 보여줘야 하기 때문입니다.

5. 왜 "통과시키면 무엇이 일어나는가"가 중요한가?

승인 버튼은 단순한 버튼이 아닙니다.

버튼을 누르는 순간 실제 시스템에서 행동이 발생합니다.

예를 들어:

| [승인]

을 눌렀는데 실제로는

| **310,000원이 고객 계좌로 송금**

된다면 담당자는 이것을 정확히 알고 있어야 합니다.

따라서 승인 화면에서는

| **판단 정보 + 실행 결과**

를 함께 보여주는 것이 중요합니다.

이렇게 해야 사람이 **무슨 일을 승인하는지 알고 승인**할 수 있습니다.

6. 승인과 반려만으로는 부족하다

처음 HITL을 설계할 때 흔히 이렇게 만듭니다.

[승인] [반려]

하지만 실제 업무에서는 애매한 상황이 많습니다.

예를 들어 담당자가 이렇게 판단할 수 있습니다.

| "환불은 해주되 전액은 너무 많다. 50%만 환불하자."

또는

| "답변 내용은 좋은데 고객에게 보내는 문구만 조금 수정하자."

이런 상황에서 승인/반려만 있다면 담당자는 어떻게 해야 할까요?

반려한 다음 직접 처리해야 합니다.

그러면 자동화가 깨집니다.

그래서 **네 가지 응답 방식**을 준비합니다.

7. 네 가지 사람의 응답

① 승인

AI가 만든 결과를 그대로 실행합니다.

[승인]

예:

| 310,000원 환불을 그대로 실행

② 수정 후 승인

가장 실무적으로 중요한 기능입니다.

예를 들어

| "전액 환불은 아니고 150,000원만 환불하세요."

처럼 사람이 일부 내용을 수정한 후 실행할 수 있습니다.

또는 고객에게 보낼 메시지의 문구를 수정한 뒤 발송할 수도 있습니다.

즉,

| **AI 결과를 버리는 것이 아니라 사람이 조금 수정해서 활용하는 것**

입니다.

③ 반려

AI가 만든 결과를 실행하지 않습니다.

예:

| "제조 불량으로 보기 어렵기 때문에 환불하지 않음."

이때는 **반려 사유도 함께 기록하는 것이 중요합니다.**

왜냐하면 나중에

- 어떤 이유로 반려했는지
- AI가 어떤 판단을 잘못했는지
- 멈춤 기준이 적절했는지

를 분석할 수 있기 때문입니다.

즉 반려 사유는 단순한 기록이 아니라 **에이전트 개선을 위한 데이터**가 됩니다.

④ 다시 판정

사람이 직접 결론을 내리는 대신 AI에게 다시 판단하도록 요청할 수도 있습니다.

예:

└ "부분 환불 기준으로 다시 계산해."

그러면 AI가 새로운 조건을 반영해 다시 판단합니다.

즉,

```
AI 판단
↓
사람 검토
↓
다시 판정 요청
↓
AI 재판단
↓
사람 승인
```

이라는 순환 구조가 가능합니다.

8. 네 가지 응답을 설계할 때 반드시 정해야 하는 것

버튼만 만드는 것으로 끝나지 않습니다.

각 버튼을 눌렀을 때 **다음에 무슨 일이 일어나는지**까지 설계해야 합니다.

예를 들어:

사람의 선택	다음 행동
승인	그대로 실행
수정 후 승인	수정된 내용으로 실행
반려	실행하지 않고 반려 기록
다시 판정	AI가 수정된 조건으로 재판단

특히 다음 질문을 미리 정해야 합니다.

수정 후 승인

수정된 금액이나 문구는 어디에 저장하는가?

반려

그냥 종료하는가, 담당자에게 다시 보내는가?

다시 판정

AI가 무한히 다시 판단하지 못하도록 횟수 제한을 둘 것인가?

예를 들어

최대 3회까지만 재판정

처럼 제한할 수 있습니다.

9. "다시 판정"은 특히 주의해야 한다

AI에게 다시 판단하라고 하면 계속 반복될 가능성이 있습니다.

```
판정
↓
다시 판정
↓
다시 판정
↓
다시 판정
↓
...
```

이런 구조는 피해야 합니다.

따라서 실제 에이전트에서는

재판정 횟수

를 상태값으로 관리하고 상한을 두는 것이 좋습니다.

예:

```
재판정 횟수 = 0
↓
다시 판정
↓
재판정 횟수 = 1
↓
다시 판정
↓
재판정 횟수 = 2
```

↓
최대 3회 도달
↓
사람에게 직접 처리 요청

이처럼 자동화에도 탈출구와 종료 조건이 필요합니다.

10. 또 하나의 중요한 문제 — 승인 대기 시간

지금까지는 사람이 바로 승인한다고 생각했습니다.

하지만 현실은 그렇지 않습니다.

담당자가

- 회의 중일 수도 있고
- 자리를 비웠을 수도 있고
- 퇴근했을 수도 있고
- 휴가 중일 수도 있습니다.

따라서 승인 대기 시간은 몇 초가 아니라 몇 시간 또는 며칠이 될 수도 있습니다.

여기서 챕터 02에서 배운 **체크포인트**가 다시 등장합니다.

11. **InMemorySaver** 만 사용하면 생기는 문제

실습에서는

```
InMemorySaver()
```

를 사용했습니다.

이것은 데이터를 프로그램의 **메모리에 저장**합니다.

따라서 프로그램이 실행되는 동안에는 괜찮습니다.

하지만 프로그램을 종료했다가 다시 실행하면 저장했던 대기 상태가 사라질 수 있습니다.

예를 들어:

31만원 환불
↓
사람 승인 대기
↓

프로그램 종료
↓
다음 날 재시작
↓
대기 상태 없음

이러면 큰 문제가 발생합니다.

고객 요청은 처리되지 않았고, 담당자도 무엇을 승인해야 하는지 알 수 없습니다.

12. 실제 서비스에서는 지속적인 저장이 필요하다

따라서 실제 서비스에서는 승인 대기 상태를 파일이나 데이터베이스에 저장해야 합니다.

앞서 배운 구조를 다시 연결하면:

AI 판단
↓
HITL
↓
interrupt
↓
체크포인트 저장
↓
승인 대기
↓
프로그램 재시작
↓
저장된 상태 복원
↓
사람 승인
↓
resume
↓
계속 실행

이렇게 해야 몇 시간 뒤, 심지어 다음 날에도 작업을 이어갈 수 있습니다.

13. 운영 단계에서는 "대기 건 관리"까지 필요하다

실제 서비스라면 단순히 상태를 저장하는 것만으로도 부족합니다.

대기 중인 요청이 계속 쌓일 수 있기 때문입니다.

예:

승인 대기
R3
R7
R12
R18
R23
R31
...

이 목록을 아무도 확인하지 않는다면 자동화가 아니라 **미처리 업무 보관함**이 됩니다.
따라서 운영 규칙도 필요합니다.

14. 승인 대기 시간이 길어지면 어떻게 할 것인가?

예를 들어

3일 동안 담당자가 처리하지 않으면 어떻게 할 것인가?

를 미리 정해야 합니다.

가능한 방법은 여러 가지입니다.

① 자동 승인

일정 시간이 지나면 자동으로 처리합니다.

② 자동 반려

기한을 넘기면 처리하지 않고 종료합니다.

③ 다른 담당자에게 전달

담당자가 처리하지 않으면 다른 사람에게 넘깁니다.

④ 다시 알림

담당자에게 반복적으로 알림을 보냅니다.

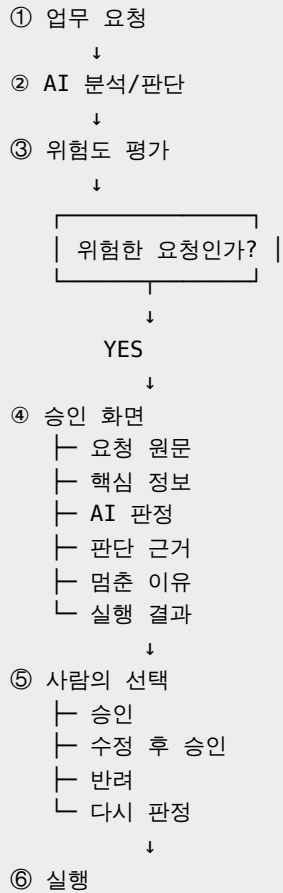
중요한 것은 특정 방법 자체가 아니라

"대기 상태가 영원히 남아 있지 않도록 운영 규칙을 미리 정한다."

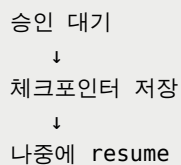
는 것입니다.

15. 이번 챕터에서 배우는 HITL의 전체 구조

앞의 챕터들과 연결하면 이제 전체 그림이 보입니다.



그리고 사람이 바로 응답하지 않을 수 있으므로



까지 연결됩니다.

16. 이번 챕터의 가장 중요한 설계 원칙

★ 원칙 1. 사람에게 다시 조사시키지 말 것

HITL의 목적은

■ **AI가 정리한 정보를 사람이 빠르게 판단하는 것**

이지,

■ **사람이 AI가 한 일을 다시 확인하는 것**

이 아닙니다.

★ 원칙 2. 승인 버튼의 결과를 명확히 보여줄 것

"승인"이라는 말만 보여주지 말고

■ **"승인하면 310,000원이 고객 계좌로 환불됩니다."**

처럼 실제 결과를 보여줍니다.

★ 원칙 3. 승인/반려만 제공하지 말 것

실무에서는

■ **승인 / 수정 후 승인 / 반려 / 다시 판정**

네 가지가 유용합니다.

★ 원칙 4. 사람의 선택도 데이터로 남길 것

특히 반려 사유나 수정 내용은 향후

- AI 판단 개선
- 멈춤 기준 개선
- 프롬프트 개선
- 업무 프로세스 개선

에 사용할 수 있습니다.

즉 HITL은 사람을 넣는 것에서 끝나는 것이 아니라 사람의 판단을 다시 시스템 개선에 활용하는 순환 구조가 될 수 있습니다.

★ 원칙 5. 승인 대기 상태를 영구적으로 보존할 것

사람의 응답은 즉시 오지 않을 수 있습니다.

따라서 실제 서비스에서는 메모리가 아닌 **지속 가능한 저장소**를 사용해야 합니다.

17. 사용자님의 AI 컨설팅 관점에서 중요한 부분

사용자님이 앞으로 기업의 AI 자동화 컨설팅을 한다면, 고객에게 단순히

| "AI가 환불 업무를 자동화합니다."

라고 설명하는 것보다 훨씬 구체적으로 설계할 수 있습니다.

예를 들어:

| **AI가 환불 요청을 분석하고 규정을 찾아 환불안을 작성합니다. 일정 금액 이상이거나 위험 표현이 포함된 건만 담당자에게 전달합니다. 담당자는 요청 원문, 금액, AI 판단, 규정 근거, 실행 결과를 한 화면에서 확인하고 10초 안에 승인·수정·반려·재판정을 선택할 수 있습니다.**

이것이 바로 **업무 프로세스 전체를 설계하는 AI 컨설팅**입니다.

AI 모델만 잘 만드는 것이 아니라

| **AI → 위험 판단 → 사람 → 승인 → 실행 → 기록 → 개선**

이라는 전체 업무 시스템을 설계하는 것입니다.

18. 이번 챕터 핵심 요약

개념	핵심 의미
승인 화면	사람이 빠르게 판단하는 인터페이스
10초 목표	사람이 추가 조사 없이 결정할 수 있도록 설계
요청 원문	AI 요약만 믿지 않고 원래 요청 확인
판단 근거	AI가 왜 그렇게 판단했는지 확인
멈춘 이유	왜 사람이 개입해야 하는지 설명
실행 결과	승인하면 실제로 무엇이 발생하는지 표시
4가지 응답	승인 / 수정 후 승인 / 반려 / 다시 판정
체크포인트	장시간 승인 대기 상태 보존
운영 규칙	오래 대기한 요청의 처리 방법 결정

 이번 챕터에서 반드시 기억할 한 문장

좋은 HITL은 사람에게 일을 다시 시키는 시스템이 아니라, AI가 판단에 필요한 정보를 한 화면에 정리해 주고 사람이 짧은 시간 안에 최종 결정하도록 만드는 시스템입니다.

Chapter05

/강의정리 챕터 05 — 사람이 승인하는 에이전트 만들기 프로젝트

평소 분량의 약 70%

1. 이번 프로젝트의 핵심

이번 챕터부터는 지금까지 배운 HITL(Human-in-the-Loop) 구조를 실제 업무에 적용합니다.

핵심은 단순히 “AI가 일을 잘하게 만드는 것”이 아닙니다.

AI가 대부분의 일을 자동으로 처리하되, 위험한 순간에는 사람에게 판단을 맡기고, 승인을 받으면 다시 자동으로 이어지는 업무 시스템을 만드는 것입니다.

즉,

업무 요청 → AI 처리 → 위험 여부 판단 →

- 안전하면 → 자동 실행
- 위험하면 → 사람 승인 → 다시 실행

이라는 구조입니다.

2. 이번 프로젝트에서 가장 중요한 관점

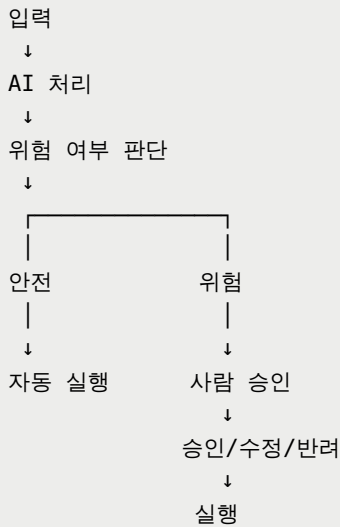
이번 과제는 겉으로 보면 “HITL 에이전트 하나 만들어 보기”이지만, 실제로 배우는 것은 더 큼니다.

기존 자동화

입력
↓
AI 처리
↓
결과

↓
자동 실행

이번 프로젝트



여기서 중요한 것은 **AI가 모든 것을 결정하는 구조가 아니라는 것**입니다.

AI는 **자동화 담당자**, 사람은 **최종 판단자** 역할을 합니다.

3. 주제를 고르는 방법

이번 프로젝트에서는 환불 승인은 제외하고, 본인이 원하는 업무를 선택합니다.

예를 들어 다음과 같습니다.

업무	사람이 개입하는 순간	위험 기준 예시
고객 문의 자동 답변	답변 발송 전	확신도 0.7 미만
콘텐츠 자동 발행	발행 전	출처 없는 수치 포함
회의록 → 할 일 등록	업무 등록 전	담당자 불명확
재고 자동 발주	발주서 전송 전	100만 원 초과
문서 자동 수정	원본 반영 전	20줄 이상 수정
일정 자동 확정	초대 발송 전	외부 참석자 포함
지원서 1차 분류	결과 통보 전	불합격 분류

여기서 중요한 것은 **업무 자체보다 "사람이 언제 개입해야 하는가"를 명확하게 정의할 수 있는 업무인지**입니다.

4. 좋은 프로젝트 주제를 고르는 기준

좋은 주제는 다음 세 가지를 갖습니다.

① AI가 상당 부분 자동화할 수 있어야 합니다.

예를 들어 고객 문의라면

문의 읽기
↓
문의 유형 분류
↓
관련 규정 검색
↓
답변 작성

까지는 AI가 할 수 있습니다.

② 마지막에 실제 실행 행위가 있어야 합니다.

예를 들어

답변 생성
↓
고객에게 이메일 발송

에서 **이메일 발송**이 외부 실행입니다.

③ 실수했을 때 사람이 개입할 이유가 있어야 합니다.

예를 들어 단순 문의는 자동으로 보내도 되지만,

“환불 규정에 예외가 적용되는지 모르겠다.”

와 같은 경우에는 사람이 확인해야 합니다.

따라서 좋은 HITL 프로젝트는

“AI가 잘할 수 있는 일 + 실수하면 문제가 되는 마지막 행동”

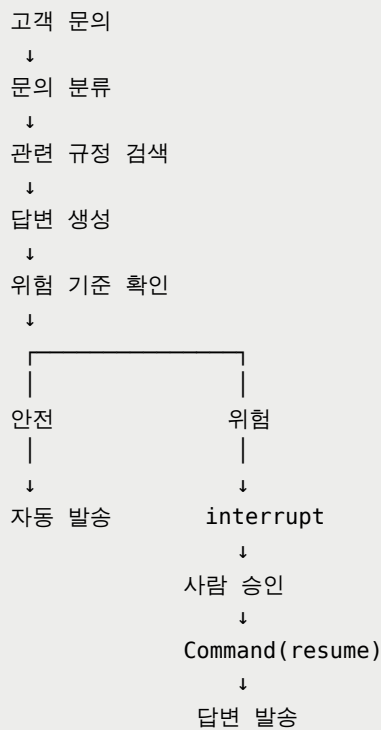
의 조합이라고 생각하시면 쉽습니다.

5. 필수 구현 ① — HITL 파이프라인

이번 프로젝트의 가장 중요한 구현 요소입니다.

핵심은 **결과를 외부로 보내기 직전에 멈추는** 것입니다.

예를 들어 고객 문의 자동 답변이라면:



여기서 **interrupt** 는 “잠깐 멈추고 사람의 답을 기다려라”라는 의미입니다.

사람이 승인하면 **resume** 을 통해 다시 이어갑니다.

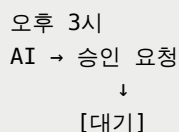
6. 체크포인트가 왜 필요한가?

사람이 바로 승인한다는 보장이 없기 때문입니다.

예를 들어 오후 3시에 AI가 승인 요청을 만들었습니다.

그런데 담당자는 회의 중이라 오후 6시에 확인합니다.

그러면 AI가 그동안 기다리고 있어야 합니다.



↓
오후 6시
사람 → 승인
↓
AI → 실행

이때 필요한 것이 **체크포인트**입니다.

체크포인트는 쉽게 말하면 **게임의 세이브 포인트**입니다.

게임을 하다가 저장해 놓으면 나중에 다시 그 지점에서 시작할 수 있듯이,

AI도 현재 상태와 어디까지 진행했는지를 저장해 두었다가 나중에 이어서 실행합니다.

따라서 이번 프로젝트에서는 단순히 **interrupt**를 사용하는 것뿐 아니라 **대기 상태를 저장하고 다시 이어가는 구조**까지 만들어야 합니다.

7. 필수 구현 ② — 승인 기준을 최소 2개 이상 정의

이번 과제에서 상당히 중요한 부분입니다.

단순히

“중요한 경우에는 사람이 승인한다.”

라고 하면 안 됩니다.

컴퓨터가 판단할 수 있는 형태로 만들어야 합니다.

예를 들어 콘텐츠 발행이라면:

기준 1

출처가 없는 통계 수치가 포함되어 있으면 승인 요청

→ 허위·검증되지 않은 정보가 외부에 나가는 위험을 막기 위한 기준

기준 2

검수 점수가 0.8 미만이면 승인 요청

→ 품질이 낮은 콘텐츠가 발행되는 위험을 막기 위한 기준
즉 기준마다 반드시 다음 질문에 답해야 합니다.

“이 기준은 어떤 위험을 막기 위해 존재하는가?”

8. 더 중요한 것 — 자동 처리되는 건도 설명해야 합니다

HITL에서는 사람에게 보내는 경우만 설명하면 부족합니다.

사람에게 보내지 않는 경우도 설명할 수 있어야 합니다.

예를 들어:

검수 점수 0.95
출처 확인 완료
민감한 내용 없음
→ 자동 발행

왜 자동 발행해도 될까요?

위험 요소가 확인되지 않았고, 사전에 정한 안전 기준을 모두 통과했기 때문입니다.

즉 HITL 설계는 사실상 두 종류의 근거를 만드는 일입니다.

사람에게 보낼 근거

“왜 이 건은 위험한가?”

자동 처리할 근거

“왜 이 건은 사람이 보지 않아도 되는가?”

이 두 가지가 모두 있어야 제대로 된 자동화 설계가 됩니다.

9. 필수 구현 ③ — 웹 데모

이번 프로젝트에서는 코드만 제출하면 안 됩니다.

사람이 직접 사용해 볼 수 있는 승인 화면을 만들어야 합니다.

Streamlit이나 Gradio 등을 사용할 수 있습니다.

화면의 핵심 구조는 다음과 같습니다.

승인 대기 목록	
요청 A	승인 대기
요청 B	승인 대기
요청 C	자동 처리 완료

↓ 요청 A 선택

요청 원문
AI 판단
판단 근거
멈춘 이유
승인하면 일어나는 일
[승인] [수정 후 승인] [반려]

이 화면은 단순히 “승인하시겠습니까?”를 보여주는 것이 아닙니다.

사람이 판단하기 위해 필요한 정보를 한 화면에 제공해야 합니다.

10. 이번 프로젝트에서 UI가 중요한 이유

앞 챕터에서 배웠듯이 HITL의 목적은 사람에게 일을 다시 떠넘기는 것이 아닙니다.

AI가 대부분의 일을 처리하고 사람은 예외적인 판단만 빠르게 하는 것입니다.

그런데 승인 화면에서 사람이 다시 여러 시스템을 찾아봐야 한다면 자동화 효과가 크게 떨어 집니다.

따라서 좋은 승인 화면은

“사람이 10초 안에 판단할 수 있도록 필요한 정보만 보여주는 화면”

이어야 합니다.

이것은 개발 문제가 아니라 서비스 기획 문제이기도 합니다.

11. REPORT.md가 중요한 이유

이번 과제의 **REPORT.md**는 단순한 실습 보고서가 아닙니다.

사실상 **AI 자동화 시스템의 설계 문서**입니다.

다음 내용을 작성해야 합니다.

① 주제와 선택 이유

무슨 업무를 자동화했는가?
왜 이 업무를 선택했는가?
어디까지 AI가 처리하는가?
어디서 외부 실행이 발생하는가?

② 파이프라인 구조도

```
입력
↓
분석
↓
판단
↓
위험 기준
↓
├─ 자동 실행
└─ HITL
    ↓
    사람 승인
    ↓
    실행
```

그리고 **State**가 어떻게 이동하는지, 어디서 멈추고 어떻게 재개되는지까지 표현해야 합니다.

③ 승인 기준과 이유

단순히 기준만 나열하지 않고,

기준 → 막으려는 위험 → 자동 처리 가능한 사례

까지 설명해야 합니다.

④ 실행 결과

예:

총 30건 처리
자동 처리 24건
승인 요청 6건

6건 중
승인 4건
수정 후 승인 1건
반려 1건

이런 식으로 실제 결과를 보여주는 것입니다.

⑤ 데모 화면

어떤 정보를 보여주고 어떤 정보를 뺐는지 설명합니다.

⑥ 회고

가장 중요한 부분 중 하나입니다.

“코드를 만들면서 가장 어려웠던 것이 무엇이었는가?”

뿐만 아니라,

“실제 서비스라면 무엇을 더 보완해야 하는가?”

까지 생각해야 합니다.

12. 이번 프로젝트를 한 단계 깊게 이해하기

이번 프로젝트의 진짜 목표는 HITL 코드를 작성하는 것 자체가 아닙니다.

지금까지 배운 것을 하나의 업무 시스템 설계 능력으로 연결하는 것입니다.

- ① 업무를 선택한다
↓
- ② AI가 어디까지 할지 정한다
↓
- ③ 외부 실행 지점을 찾는다
↓
- ④ 위험을 정의한다
↓
- ⑤ 위험 기준을 계산 가능하게 만든다
↓
- ⑥ 위험한 경우만 사람에게 보낸다
↓
- ⑦ 사람이 빠르게 판단할 화면을 만든다
↓
- ⑧ 승인 후 자동 실행한다
↓
- ⑨ 결과를 측정하고 기준을 개선한다

이 흐름을 이해하면 단순한 **AI 사용자**에서 **AI 업무 프로세스를 설계하는 사람**으로 한 단계 올라갑니다.

13. 사용자님의 AI 컨설팅 목표와 연결하면

이 프로젝트는 특히 사용자님이 준비하고 계신 **AI 컨설팅**과 연결성이 높습니다.

기업에서 흔히 하는 질문은

“이 업무를 AI로 자동화할 수 있나요?”

입니다.

그런데 실제 컨설팅에서는 여기서 끝나면 안 됩니다.

더 중요한 질문은 다음과 같습니다.

- AI가 어디까지 자동화할 것인가?
- 어디서 사람이 확인해야 하는가?
- 어떤 상황을 위험하다고 볼 것인가?
- 승인자는 어떤 정보를 봐야 하는가?
- 승인하지 않으면 어떻게 되는가?
- 승인 대기 상태는 어떻게 관리하는가?
- 사람이 잘못 승인하면 어떻게 되는가?
- 실행 결과를 어떻게 기록할 것인가?

즉 앞으로 사용자님이 기업에 AI 자동화를 컨설팅할 때는

“AI를 붙여드립니다.”

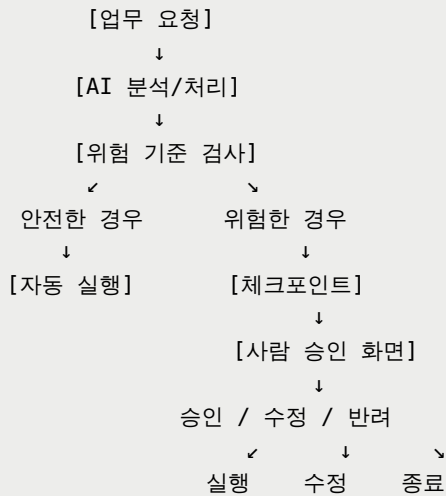
보다

“AI가 대부분의 업무를 자동 처리하되, 위험한 업무만 사람에게 넘기고 승인 후 다시 실행 되는 구조로 설계해드립니다.”

라는 관점이 훨씬 중요합니다.

14. 이번 프로젝트의 핵심 구조

전체를 하나의 그림으로 기억하시면 됩니다.



이것이 이번 프로젝트에서 만드는 “사람이 승인하는 에이전트”의 기본 형태입니다.

15. 꼭 기억해야 할 것 TOP 6

① HITL의 목적은 사람에게 일을 맡기는 것이 아닙니다.

AI가 대부분 처리하고 위험한 순간만 사람이 판단하는 것입니다.

② 승인 지점은 외부 실행 직전입니다.

메일 발송, 발주, 게시, 등록, 일정 확정처럼 되돌리기 어려운 행동 직전에 둡니다.

③ 승인 기준은 계산 가능해야 합니다.

“중요하면 승인”이 아니라

“금액 > 100만 원”, “확신도 < 0.7”처럼 표현해야 합니다.

④ 자동 처리의 근거도 필요합니다.

“왜 사람이 보지 않아도 되는가?”까지 설명할 수 있어야 합니다.

⑤ 승인 화면은 업무 효율의 일부입니다.

좋은 HITL은 좋은 AI뿐 아니라 사람이 빠르게 판단할 수 있는 UI까지 포함합니다.

⑥ 이번 프로젝트의 진짜 산출물은 코드가 아니라 설계 능력입니다.

업무 → 위험 → 승인 기준 → 사람 판단 → 자동 실행의 전체 구조를 설계하는 능력을 배우는 것입니다.

한 문장으로 정리

이번 프로젝트는 AI가 모든 것을 자동으로 실행하게 만드는 것이 아니라, AI가 대부분의 업무를 처리하고 위험한 순간에는 사람의 판단을 끼워 넣어 안전하게 다시 자동 실행하는 '현실적인 AI 업무 시스템'을 만드는 실습입니다.